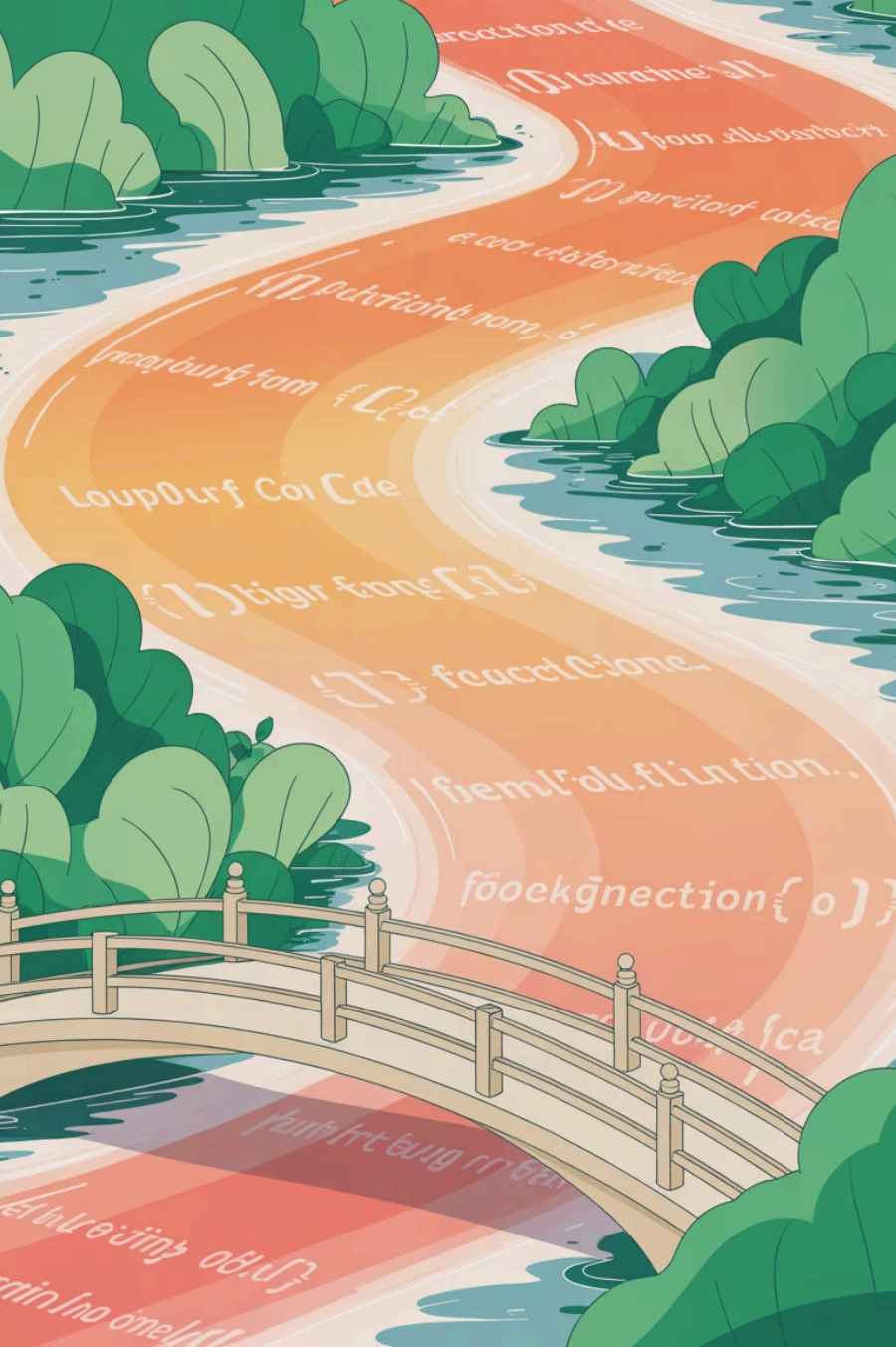
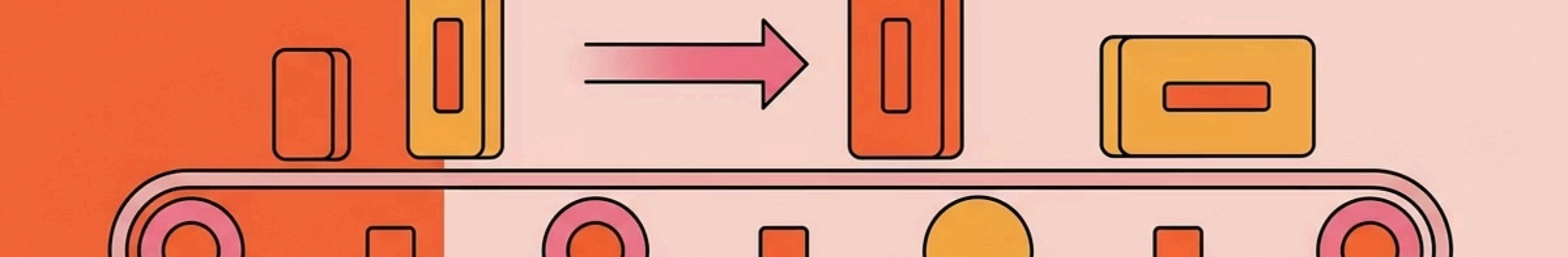




# ¿Qué es un Stream en Java?

Entendiendo la herramienta clave para programar de forma funcional

De lo imperativo a lo funcional: cómo fluyen los datos en Java



# Un Stream es...

Una secuencia de elementos que se procesan de forma **declarativa**. No almacena datos, sino que los recorre y transforma.



## Vista sobre los datos

No es una colección, sino una vista que permite operar sobre los elementos sin modificar la fuente original.



## Flujo unidireccional

Los elementos fluyen en una sola dirección, desde la fuente hasta las operaciones terminales.



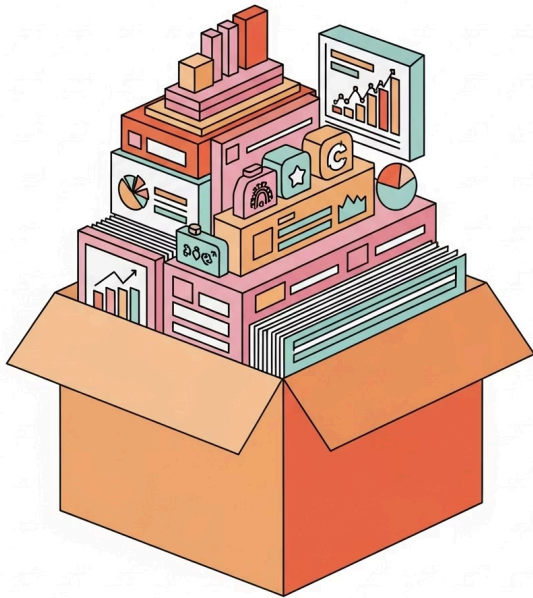
## Operaciones encadenadas

Permite conectar múltiples operaciones de forma fluida y legible.

# Stream ≠ Colección

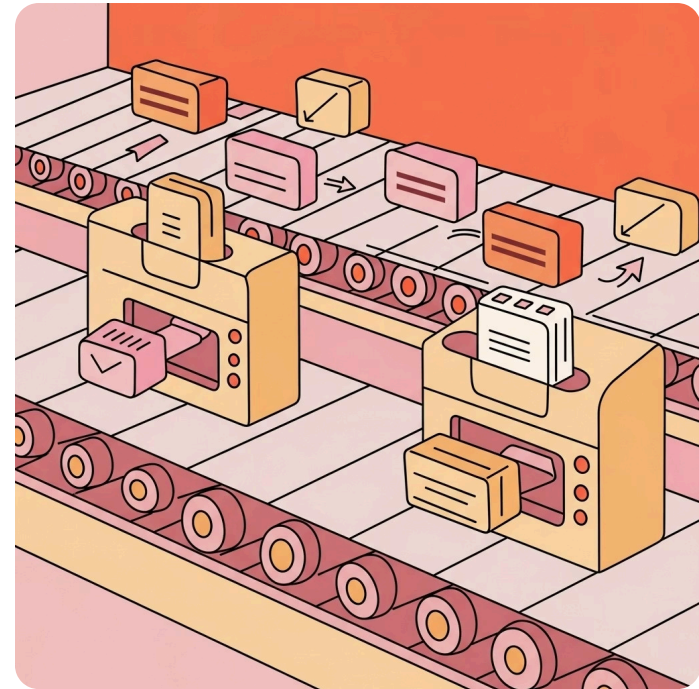
Aunque ambos trabajan con datos, su enfoque y comportamiento son fundamentalmente diferentes:

## Colección



- Almacena físicamente los elementos
- Se puede recorrer múltiples veces
- Acceso aleatorio a cualquier elemento
- Es como una **caja** que guarda objetos

## Stream

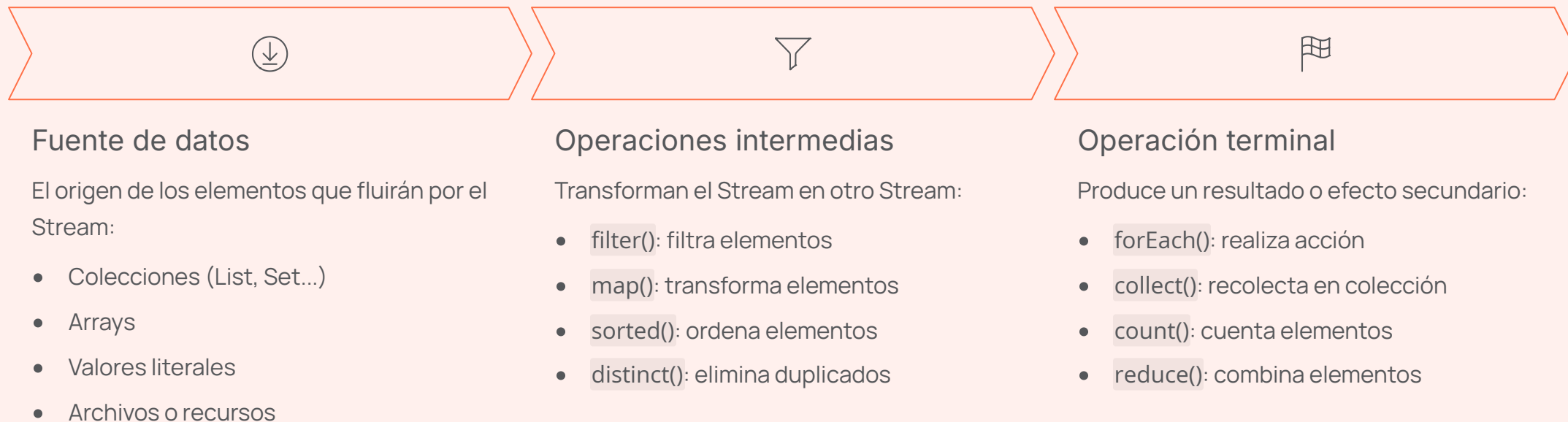


- No almacena, sino que procesa al vuelo
- Se consume al usarse, no se puede reiniciar
- Acceso secuencial, en una dirección
- Es como una **cinta transportadora** en movimiento

📌 Si intentas recorrer un Stream que ya ha sido consumido, recibirás una excepción: `IllegalStateException`.

# Cómo fluye un Stream

Un Stream funciona como una cadena de procesamiento (pipeline) con tres etapas bien definidas:



Nada se ejecuta hasta que llega la operación terminal — esto se conoce como **evaluación perezosa** (lazy evaluation).



# Formas de crear un Stream

Java ofrece varias maneras de crear Streams según la fuente de datos que tengamos disponible:

1

## Desde una colección

Cualquier colección (List, Set, etc.) puede convertirse en Stream:

```
List lista = Arrays.asList("a", "b", "c");  
Stream stream = lista.stream();
```

2

## Desde un array

Los arrays pueden convertirse usando la clase Arrays:

```
String[] array = {"a", "b", "c"};  
Stream stream =  
Arrays.stream(array);
```

3

## Directamente de valores

Puedes crear un Stream a partir de valores específicos:

```
Stream stream = Stream.of("a", "b", "c");
```



# Lo que aprendimos hoy

## 1 Qué es un Stream

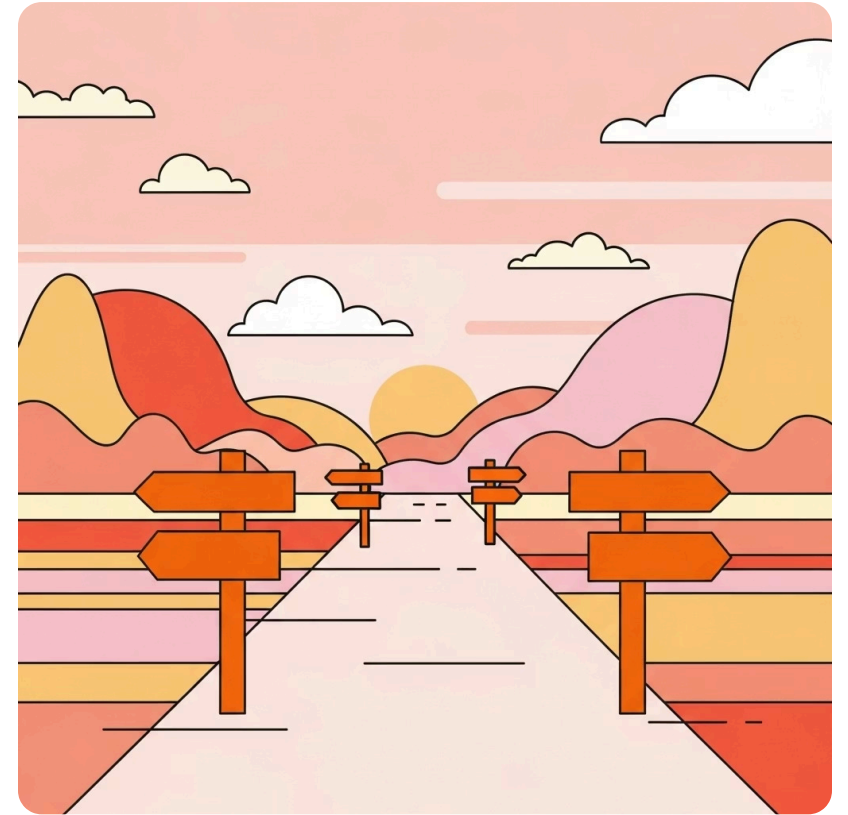
Una secuencia de elementos que se procesan de forma declarativa, permitiendo un estilo de programación funcional en Java.

## 2 Stream vs Colección

Las colecciones almacenan, los Streams procesan. Uno es un contenedor, el otro es una cinta transportadora.

## 3 El pipeline de un Stream

Fuente → Operaciones intermedias → Operación terminal, con evaluación perezosa que optimiza el procesamiento.



En el próximo video:

Expresiones lambda: la clave para escribir código funcional con Streams